

IEEE STD 1016-2009 STANDARDS SPECIFICATION

Software Design Document (SDD)

Crypto Crime Intelligence Tool (CCIT)

Algorithmic Architecture, Graph Systems, and Explainable Money-Laundering Typology Detection

Architectural Blueprint: This Software Design Document establishes the complete system architecture, data pipelines, graph algorithms, and cryptographic verification mechanisms for the **Crypto Crime Intelligence Tool (CCIT)**. CCIT implements Section 4 of the national Police-AI architectural framework by translating raw, multi-chain public ledger broadcasts into unified graph models, resolving entity clusters via Union-Find and temporal sweep heuristics, detecting money-laundering typologies (peel chains, rapid pass-throughs, smurfing/structuring, and mixer hops), and generating court-admissible forensic intelligence dossiers with strict **Human-In-The-Loop (HITL)** oversight.

SYSTEM CODENAME

CCIT / Project Cybersleuth

PRIMARY ARCHITECTURAL PATTERN

Event-Driven Graph Analytics Pipeline

GRAPH & RELATIONAL ENGINES

Neo4j Property Graph / PostgreSQL 15+ / Redis

DOCUMENT VERSION

1.0.0 (Design Baseline)

FRAMEWORK ALIGNMENT

Section 4 Architecture (UC-81, UC-80, UC-83)

EVIDENCE STANDARD

ISO/IEC 27037 (Digital Forensics Chain of Custody)

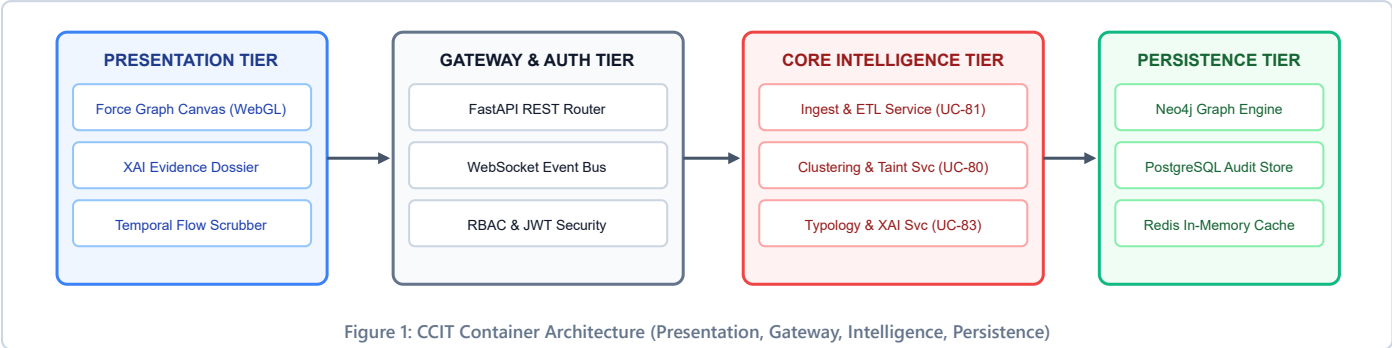
Technical Guarantee: The architecture specified herein is strictly non-custodial and operates exclusively on public distributed ledger data. It provides mathematical explainability (XAI) for all anomaly scores and enforces mandatory human investigator verification prior to any case lead export or judicial filing.

1 System Architecture & Subsystem Decomposition

1.1 Architectural Pattern & Police-AI Framework Synthesis

The CCIT is engineered as a modular, high-throughput financial intelligence pipeline. It operationalizes Section 4 of the national Police-AI standard by serializing three distinct police use cases into a unified data flow: **UC-81** (Ingestion & Graph Normalization), **UC-80** (Entity Clustering & Multi-Hop Taint), and **UC-83** (Typology Pattern Recognition & XAI Scoring).

1.2 C4 Container Architecture Diagram



1.3 Subsystem Responsibilities Matrix

Subsystem	Core Responsibilities & Components	Tech Stack	Framework Mapping
Ingestion Subsystem	Connects to public JSON-RPC nodes; unpacks blocks, transaction receipts, ERC-20 Transfer logs; maps inputs/outputs to canonical transfers.	Python 3.11, Web3.py, aiohttp, AsyncIO	UC-81 FOUNDATION
Graph & Clustering	Builds directed multigraphs; runs Disjoint-Set Union-Find for UTXO ownership; executes deposit-sweep clustering; computes FIFO/Poison taint.	Neo4j 5.x, NetworkX, SciPy, Cypher Engine	UC-80 ANALYSIS
Typology & XAI	Executes graph motif pattern matching for peel chains, rapid layering, and smurfing; computes normalized scores; generates natural-language cards.	NumPy, Rule Engine, Scikit-Learn, Jinja2	UC-83 INTELLIGENCE
Forensic Audit & UI	Maintains ISO 27037 chained SHA-256 logs; provides interactive WebGL graph visualization and human lead verification workflow.	React 18, Cytoscape.js, PostgreSQL, Tailwind	SECTION 4 REVIEW

2 Detailed Component & Algorithmic Design

2.1 Canonical Data Normalization (UC-81)

Multi-chain ledger transactions are standardized into a canonical transfer primitive before graph ingestion:

```
class CanonicalTransfer:
    tx_hash: str          # Unique blockchain transaction hash
    network: str          # "BTC" (UTXO), "ETH" (Account), "TRON"
    from_address: str     # Origin address or input set root
    to_address: str       # Destination address or contract
    amount_crypto: float  # Raw cryptocurrency volume
    amount_usd: float     # Historical price rate at block execution timestamp
    timestamp: datetime   # Immutable block header UTC timestamp
    fee_usd: float        # Gas / mining fee paid
    hop_depth: int        # Graph distance from target seed address
```

2.2 Entity Clustering Engine: Disjoint-Set Union (UC-80)

To resolve pseudonymous addresses into singular real-world actors, CCIT executes the **Common-Input Ownership Heuristic** via a Disjoint-Set (Union-Find) data structure with path compression and union-by-rank:

UTXO Common-Input Rule

If transaction T consumes inputs $\{I_1, I_2, \dots, I_m\}$, all inputs must have been co-signed by the private keys of the same entity. Hence:

$$\text{Cluster}(I_1) = \text{Cluster}(I_2) = \dots = \text{Cluster}(I_m)$$

Executed with time complexity $O(\alpha(N))$ where α is the inverse Ackermann function.

EVM Temporary Sweep Rule

An Account-based address A_{temp} is clustered with consolidation address A_{target} if:

$$\text{Lifetime Tx's} < 5 \quad | \quad \Delta t_{\text{hold}} \leq 300 \text{ s} \quad | \quad \text{Volume Dispersed} \geq 98.5\%$$

Eliminates ephemeral intermediary transit wallets from obscuring true custody.

2.3 Money-Laundering Typology Detection Engines (UC-83)

Typology	Detection Algorithm & Graph Motif	Mathematical Parameters	Weight (w_k)
Peel Chain	Directed DFS path search identifying sequential transfers where the majority balance is repeatedly forwarded to a new address while small fractions are peeled off.	Path Length $L \geq 5$ Forward > 90% Peel < 10%	$w_1 = 0.35$
Rapid Pass-Through (Layering)	Temporal flow analysis identifying wallets receiving large inflows and executing immediate outflow with zero balance retention.	Holding $\Delta t < 600 \text{ s}$ Balance < \$10 USD Velocity > 95%	$w_2 = 0.30$
Smurfing / Structuring	Bipartite subgraph pattern matching detecting 1-to-N dispersion (Fan-Out) followed by N-to-1 aggregation (Fan-In) within a constrained time window.	Fan-Out $N \geq 4$ Window $W \leq 48 \text{ h}$ Volume Conserved $\geq 90\%$	$w_3 = 0.40$
Mixer & Bridge Hop	Direct edge evaluation cross-referencing smart contract addresses against known anonymizers (e.g. Tornado Cash) or unverified cross-chain bridges.	Known Entity Match Hops ≤ 2	$w_4 = 0.50$

2.4 Composite Suspicion Formula & Explainable AI (XAI) Generation

For any active entity cluster C, the normalized Suspicion Score $S \in [0, 100]$ is calculated as:

$$S(C) = \min(100, \quad 100 \times [\sum (w_k \times f_k(C))] \times [1 + \text{Taint}_{\text{poison}}(C)] \quad)$$

Where $f_k(C) \in [0, 1]$ represents the activation of typology feature k. The engine translates activations into natural-language **Evidence Cards**:

Generated Evidence Card Example (Rapid Pass-Through Layering)

"Entity Cluster #04 (0x7a2...48f) flagged with Suspicion Score 88/100 (Confidence: 94%). Inbound transfer of 45.0 ETH was forwarded across 3 intermediary hops within 145 seconds ($\Delta t < 600\text{s}$), retaining 0.00 ETH balance. Taint propagation traces 100% of funds to illicit origin hash 0x9c3...12e. Recommended Action: Subpoena exchange deposit wallet 0x3d1...89a."

3 Data Architecture & API Specifications

3.1 Neo4j Property Graph & PostgreSQL Schemas

Neo4j Property Graph Primitives

```
(:WalletAddress {address, network, balance_usd, label, risk_score})
(:EntityCluster {cluster_id, heuristic, wallet_count, net_vol})
(:WalletAddress)-[:BELONGS_TO]->(:EntityCluster)
(:WalletAddress)-[:TRANSFER {tx_hash, asset, usd_val, timestamp}]->(:WalletAddress)
(:TypologyAlert)-[:FLAGS_ENTITY]->(:EntityCluster)
```

PostgreSQL Audit & HITL Tables

```
CREATE TABLE chain_of_custody_audit_log (
  log_id BIGSERIAL PRIMARY KEY, timestamp TIMESTAMPTZ DEFAULT NOW(),
  investigator_id UUID NOT NULL, action_type VARCHAR(64) NOT NULL,
  resource_id VARCHAR(128), query_payload JSONB, sha256_checksum CHAR(64), previous_log_hash CHAR(64)
);
CREATE TABLE alert_reviews (
  review_id UUID PRIMARY KEY, alert_id VARCHAR(64), case_id UUID,
  decision VARCHAR(32) CHECK (decision IN ('LEAD', 'FP', 'INCONCLUSIVE')),
  justification TEXT NOT NULL, reviewed_at TIMESTAMPTZ DEFAULT NOW()
);
```

3.2 Core RESTful API Endpoints

Endpoint	Method	Request Parameters	Response Structure
/api/v1/investigate/trace	POST	seed_address, blockchain, max_hops, start_date, end_date	subgraph_id, nodes_count, edges_count, execution_ms
/api/v1/typologies/scan	POST	subgraph_id, typology_filter[], min_suspicion	alerts[], evidence_cards[], scanned_nodes
/api/v1/alerts/{id}/verify	POST	case_id, decision, justification, warrant_ref	status: RECORDED, chained_hash, audit_id
/api/v1/cases/{id}/export	GET	case_id, format: "pdf" "json-ld"	Binary Application/PDF + X-Forensic-SHA256

3.3 Human-In-The-Loop Interactive Dashboard Wireframe

```
+-----+
| [LOGO] CCIT - Crypto Crime Intelligence Tool | Case: #2026-CR-8902 | Detective: J. Vance (Badge #401) |
+-----+
| CONTROLS: [ Seed: 0x904b... | ETH | Hops: 4 | Scan] | METRICS: Nodes: 64 | Txs: 112 | Flagged Leads: 2 |
+-----+
| FORCE-DIRECTED GRAPH WORKSPACE | INSPECTOR & EVIDENCE CARD DRAWER |
| (Mixer Hop) | Selected Entity: Cluster #04 (3 Wallets) |
| / | Suspicion Gauge: [===== 88/100 =====] HIGH |
| [Seed] ==> [Pass-Thru Wallet] ==> [Exchange] | EVIDENCE SUMMARY (XAI): |
| \ | "Rapid Pass-Through layering detected. 98.4% |
| (Retail Wallet) | of 45 ETH moved within 145s across 3 hops." |
| | ----- |
| Legend: | HUMAN DECISION (HITL VERIFICATION): |
| * Green: Clean Retail (<30) | [ CONFIRM AS LEAD ] [ DISMISS (FALSE POS) ] |
| * Red: High Suspicion (>60) | Mandatory Justification / Warrant Reference: |
| * Purple: Attributed VASP/Bridge | [ Warrant #2026-W-401 target wallet match... ] |
+-----+
| TIMELINE PLAYBACK: |<< [Play] >>| ===== [2026-08-12 14:22 UTC] |
+-----+
```

4

Security, Forensic Standards & Formal Sign-Off

4.1 Forensic Chain of Custody Protocol (ISO 27037)

To satisfy legal admissibility requirements in criminal proceedings, CCIT implements a cryptographically sealed chain-of-custody protocol:

- Chained Block Hashing:** Every investigative step, user query, graph state snapshot, and verification decision is hashed sequentially into an immutable audit chain:

Hash_n = SHA-256(Hash_{n-1} || Timestamp || InvestigatorID || ActionType || JSON_Payload)
- Evidentiary PDF Sealing:** When an intelligence dossier is exported, the backend generates an authoritative SHA-256 checksum over the raw PDF byte stream and embeds the digest into the HTTP response header and system audit log. Tampering with any byte invalidates the judicial signature.

4.2 Security Controls & Access Partitioning

Security Layer	Implementation Mechanism	Regulatory Standard
Encryption at Rest	Database tables, suspect case notes, and API credentials encrypted via AES-256-GCM.	FIPS 140-2 / ISO 27001
Encryption in Transit	All browser sessions, microservice gRPC calls, and database links enforce TLS 1.3.	NIST SP 800-52 Rev. 2
Access Control (RBAC)	Role-based authorization: Analyst (Triage), Detective (Warrants), Auditor (Sign-Off).	EU AI Act Article 14

4.3 Design Verification & Acceptance Traceability

ID	Design Target	Verification Method	Acceptance Benchmark
DV-01	Multi-Chain Ingestion (UC-81)	Blockstate Parity Automated Test	100% address, value, and fee parity across 1,000 blocks.
DV-02	Union-Find Clustering (UC-80)	UTXO Historical Benchmark	Multi-input addresses grouped into cluster ID with zero false mergers.
DV-03	Typology Engines (UC-83)	Synthetic Vector Ingestion	> 95% detection recall on peel chains and rapid pass-throughs.
DV-04	Forensic Audit Integrity	Tamper Invalidation Test	Modifying 1 database bit causes chain-validation failure.

4.4 Institutional Submission & Engineering Approval

This Software Design Document formalizes the architectural baseline for the **Crypto Crime Intelligence Tool (CCIT)**. Signatures below verify that the design fulfills all requirements of the Police-AI framework.

LEAD AI / GRAPH SYSTEMS ARCHITECT

Project Cybersleuth Engineering Team

Signature: _____

Date: _____

SUPERVISORY DETECTIVE / EVALUATION CHAIR

Financial Crime & Cyber Intelligence Division

Signature: _____

Date: _____